this category of problems. In particular, we discussed two solutions of the problem, seen as *crossing-number optimization* and as a *force-directed graph drawing*; gradient descent is particularly suitable for the latter, while particularly bad for the former.

One issue we have seen with gradient descent is that it tends to get stuck in local minima, which is pretty much the last thing we would want, considering that we often have to deal with cost functions having lots of local peaks.

We have already discussed one workaround for this issue, running gradient descent several times using random-restart to choose a different starting point each time.

Still, even with this technique, each iteration of gradient descent is pretty much doomed (in the best-case scenario) to end in the closest local minimum along the steepest path from the starting point: it would be great, instead, if even a single run could have some non-null[1] probability to move over a local minimum, and find a better solution.

In this chapter, we'll discuss an algorithm that does exactly that, and more: it also overcomes some of the constraints that limit the applicability of gradient descent. After reading this chapter, you will learn about **simulated annealing**, a powerful optimization technique, how to apply it to some of the hardest problems on graphs, and weigh PROs and CONs of choosing it over other optimization techniques.

Simulated annealing is quite powerful as it is: among the algorithms we have described, it's the only one that's able to balance narrow and broad search, exploring a large portion